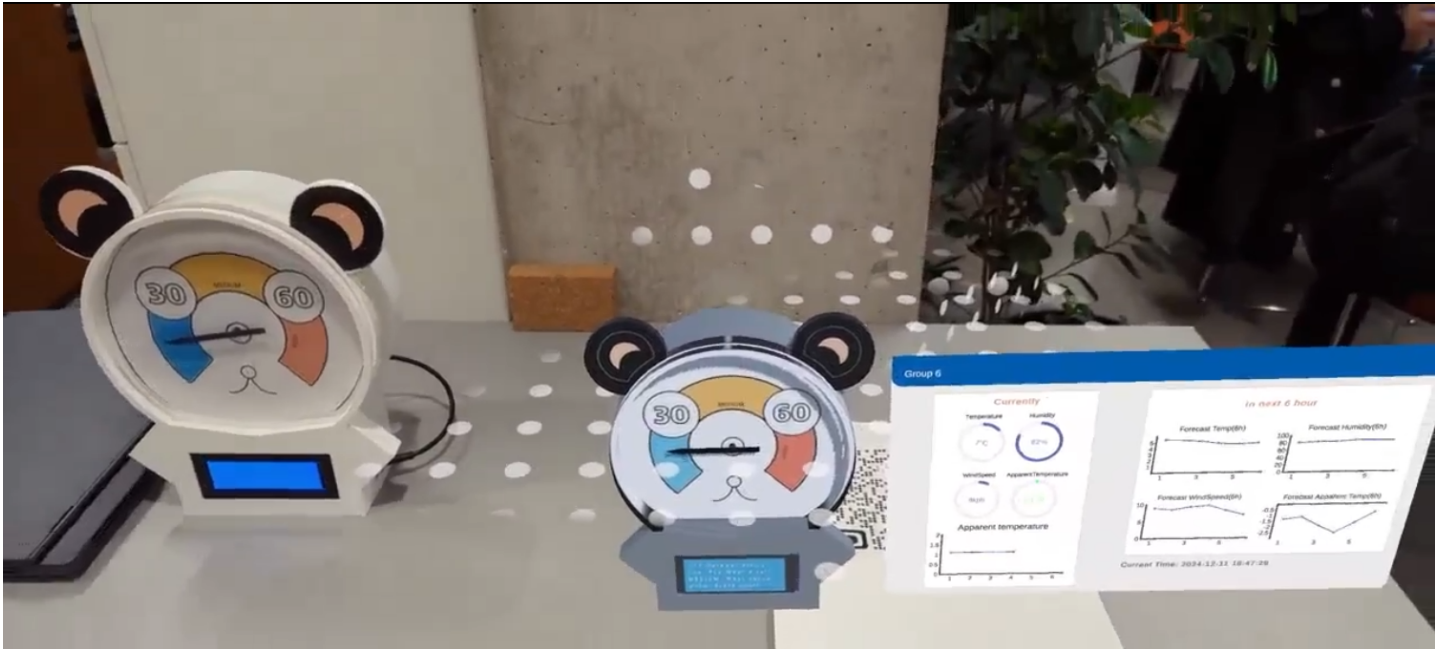


CASA0019-SDV-Group-Storm -- Final Report



Our valuable Contributors 🤖🤖 :



Dankao Chen : 24067882

Muchen Han : 24100754

Ying Wu : 23074009

Github: <https://github.com/Reikimen/CASA0019-SDV-Group-Storm>

Wondering how much clothing you need to wear before walking out of the building?

Students at UCL EAST (including my group members and I) experience this situation quite often: because the classroom is heated or air-conditioned, a fairly comfortable office environment is maintained, resulting in a sudden cold wind that freezes you into a popsicle or a sun that is about to melt you when you go out. To solve this problem, our group designed a dial that displays the outdoor body temperature as a reminder to dress and undress.

This is the URL used for showcase: <https://codepen.io/Reikimen/pen/NPKRRJR>

Background and Motivation

In daily life, people's clothing choices are often influenced by the weather. However, most tend to rely solely on the temperature indicated in weather forecasts, overlooking other environmental factors that affect how the human body actually perceives temperature. For instance, under the same temperature, higher humidity can make a person feel stuffier or colder; meanwhile, in a windy environment, even a relatively high temperature may still feel chilly due to the wind chill effect.

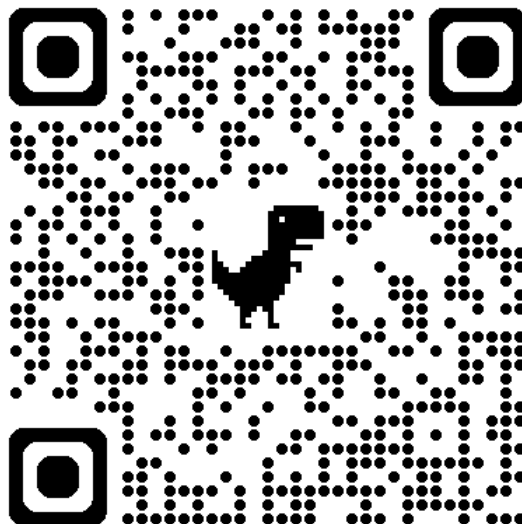
On the other hand, indoor and outdoor environments differ in terms of temperature, humidity, and wind speed. And indoor conditions can shift continually because of factors such as heating or air conditioning. For example, in winter, even if it is cold outside, turning on indoor heating may raise the indoor temperature significantly, making it uncomfortably hot if one dresses solely based on outdoor conditions. In summer, variations in air conditioning efficiency can result in some areas being quite cool while others remain relatively stuffy, making it difficult to gauge the appropriate level of clothing thickness.

A device that provides clothing guidance based on real-time data can help people avoid health issues stemming from inappropriate dress, allowing them to remain comfortable throughout different seasons and weather conditions, and better adapt to changing environments.

How you can use our product

Open the app and scan the QR code of our project GitHub, the virtual bear Gauge will appear, as well as the dashboard. You can see the virtual bear and the physical bear synchronized display, and in the dashboard there is a display of the temperature, humidity, wind speed, body temperature values. (Just as the cover shows).

Here is the relative QR code:



The Design of the Project

DataSet

This Data Device (both Physical and Digital one) uses the real time temperature, humidity and wind speed from the UCL OPS Gardon. Specifically, subscribe to the following MQTT topics:

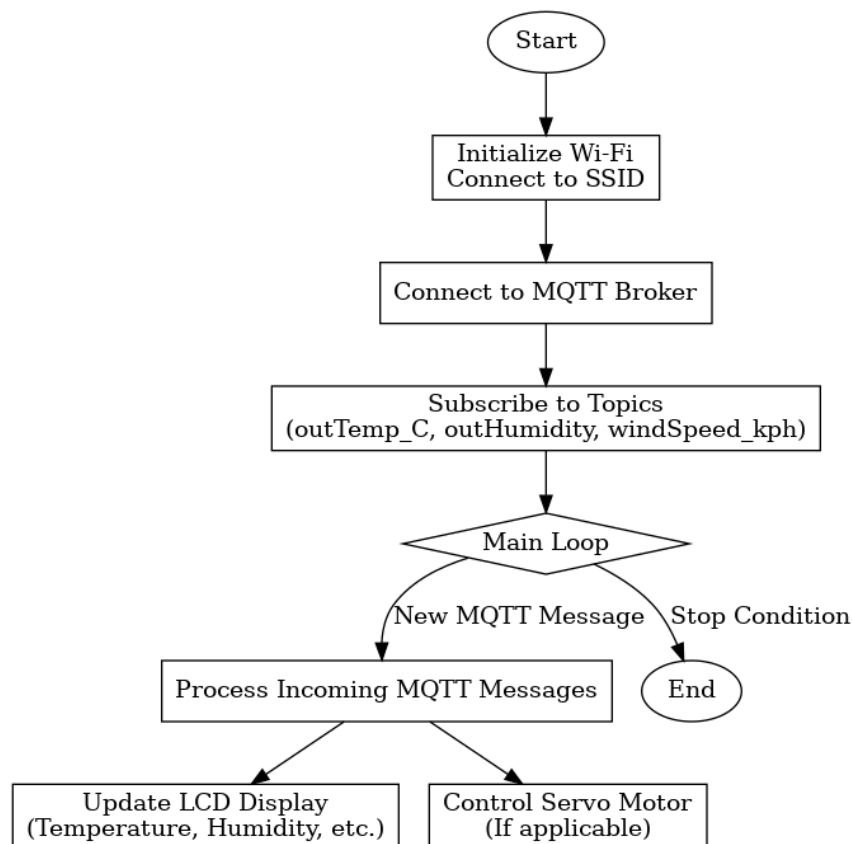
```
const char* topic1 = "UCL/OPS/Garden/WST/dvp2/outTemp_C";  
const char* topic2 = "UCL/OPS/Garden/WST/dvp2/outHumidity";  
const char* topic3 = "UCL/OPS/Garden/WST/dvp2/windSpeed_kph";
```

(The API used for weather forecast: <https://open-meteo.com/>)

DATA DEVICE (Physical)

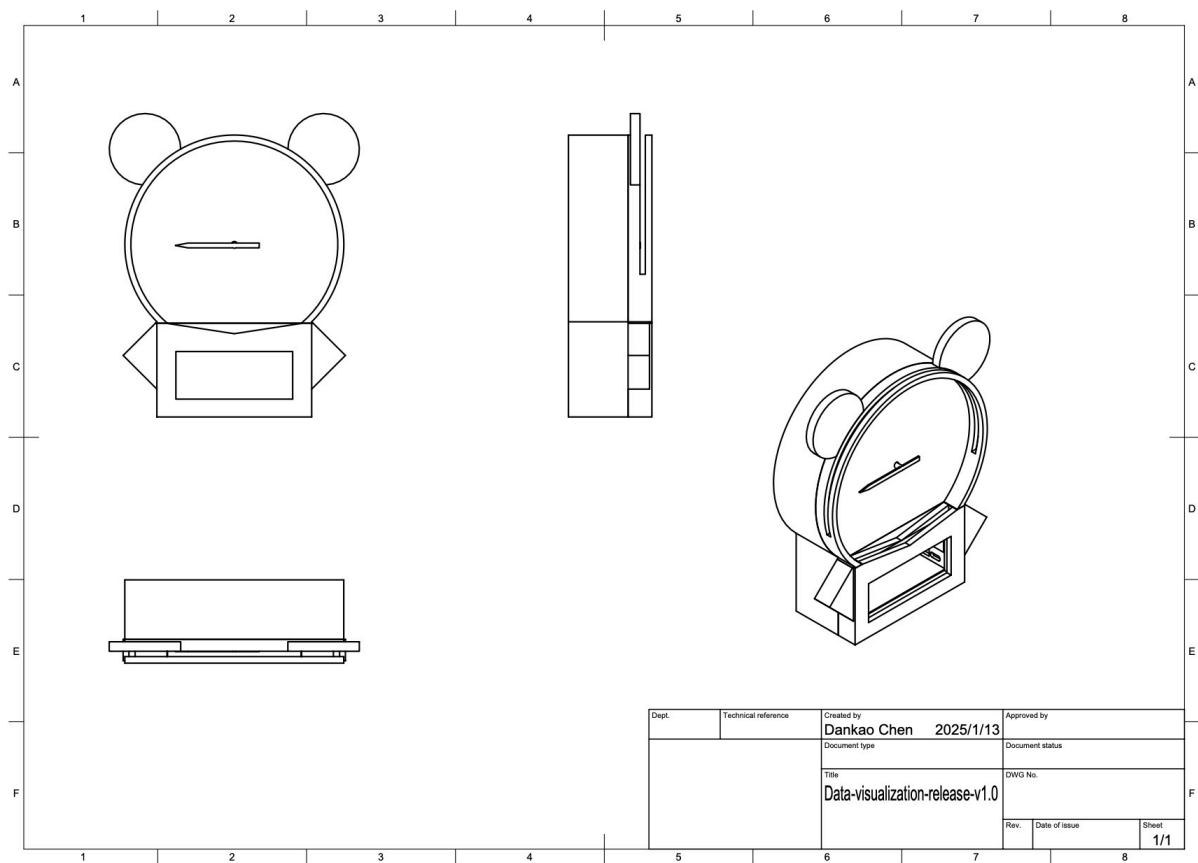
The device is in the shape of a bear and the icon indicates the body temperature index (at UCL OPS).

Device Workflow (Arduino)



The flowchart above shows the workflow of the physical part of the project: the program starts and instantiates the Wi-Fi and connects to the specified MQTT proxy, subscribes to the relevant topics and enters the main loop. In the loop, the programme processes incoming MQTT messages, updates the LCD display based on the data, and controls the servos when needed. The main loop runs continuously, but no shutdown condition is set.

Equipment structural design (Fusion)

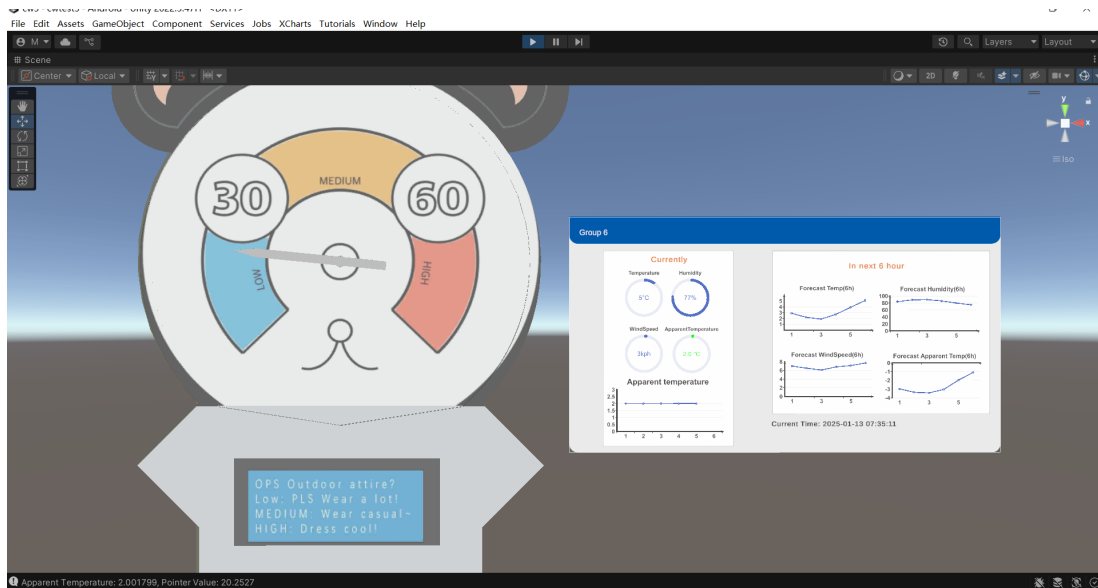


When the index is in the “LOW” zone, it means that the temperature is low and you need to wear more clothes when you go out. When the index is in the “MEDIUM” zone, it means that the temperature is moderate and you can wear a normal amount of clothes. When the index is in the “HIGH” zone, it means that it is unusually hot outside, so try to wear cooler clothes.

DATA DEVICE (Digital)

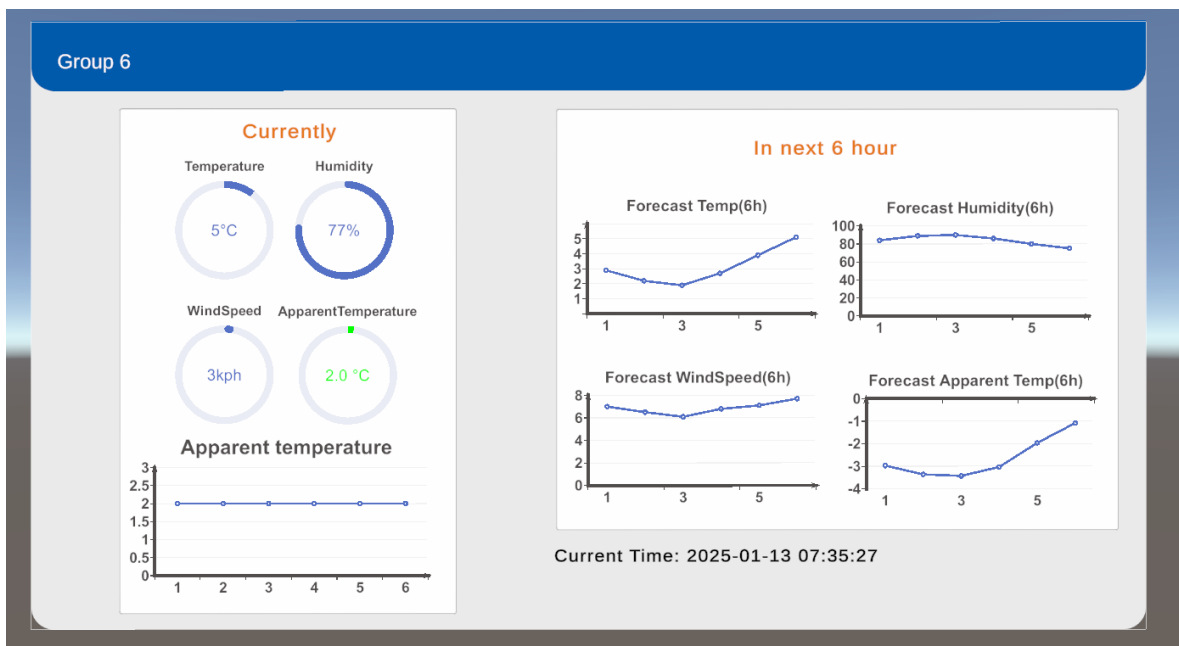
The AR part includes a dashboard and a digital version of the gauge. The digital version of the gauge performs the same function in AR as the real version, which is able to remind people of the current outdoor temperature conditions by pointing to different zones.

Views inside Unity



And the dashboard provides additional information for tracking real-time weather conditions and short-term forecasts. It provides accurate real-time weather data such as temperature, humidity, wind speed and body temperature. In addition, it provides a line graph of these data for the next 6 hours to give people a better visualisation of the day's temperature changes so that they can be well prepared for the day's attire.

Dashboard Design



Above is the example dashboard display (13 January 2025 07:35:27). The left side shows the current temperature (5°C), humidity (77%), wind speed (3km/h) and body temperature (2°C) with a graphical representation of the change in body temperature. On the right side is the weather forecast for the next 6 hours, including the trend graphs of temperature, humidity, wind speed and body temperature. The whole interface is simple and intuitive for real-time weather monitoring.

Unity into AR

In AR Bear Gauge, all interactions are controlled through a custom C# script implemented in Unity (aka mqttmcontroller). After Unity was completed, it was installed on a Google phone in CE LAB through the unity export APK feature. Using the mobile phone, scan the QR code laid flat on the desktop to display the AR bear Gauge.

How can we define the criteria for dressing?

0. Variable Used

- **OutTemp_C**: Outside temperature (Celsius).
- **WindSpeed_kph**: Wind speed (kilometers per hour).
- **outHumidity**: Relative humidity (percentage).
- **waterVaporPressure**: Water vapor pressure.

1. Calculate Water Vapor Pressure:

```
waterVaporPressure = (outHumidity / 100) * 6.105 * exp((17.27 * OutTemp_C) / (237.7 + OutTemp_C));
```

- This formula calculates the water vapor pressure in hectopascals (hPa).

2. Calculate Apparent Temperature:

```
apparentTemperature = (1.04 * OutTemp_C) + (0.2 * waterVaporPressure) - (0.65 * WindSpeed_kph) - 2.7;
```

- This formula combines temperature, humidity, and wind speed to calculate the apparent temperature.

3. Find the ups, comfortable and collars of the Apparent Temp (From Wiki: Climate of London, Greenwich Weather Station):

◦ High Temperature Condition:

```
OutTemp_C = 38.1;  
outHumidity = 100;  
WindSpeed_kph = 0;  
Apparent Temperature: 50.19
```

◦ Low Temperature Condition:

```

OutTemp_C = -9.2;
outHumidity = 0;
WindSpeed_kph = 60;
Apparent Temperature: -54.48

```

◦ Comfortable Condition:

```

OutTemp_C = 21;
outHumidity = 50;
WindSpeed_kph = 0.2;
Apparent Temperature: 21.49 -> 21.5

```

4. Calculate Apparent Temperature Index:

```

Apparent_Index = (apparentTemperature + 11.5) * 1.5;
if (Apparent_Index < 0) Apparent_Index = 0;
if (Apparent_Index > 90) Apparent_Index = 90;

```

- Converts the apparent temperature to an index range (0-90).
- Adds 11.5 to the apparent temperature, then multiplies by 1.5.
- Ensures the index is between 0 and 90.

This code calculates the water vapor pressure and apparent temperature using temperature, humidity, and wind speed data. It then converts the apparent temperature to an index, which can be used to determine the appropriate amount of clothing to wear based on current weather conditions.

Intro of Project Repo

```

CASA0019-SDV-GROUP-STORM/
├── blender/
├── Fusion/
│   ├── Glass-cover/
│   ├── Main-Box/
│   ├── Pointer/
│   ├── SG90-Holder/
│   ├── Data-visualization-release-v1.0.f3d
│   └── Data-visualization-release-v1.0.fbx
├── Image/
│   ├── Assert/
│   ├── Readme/
│   └── Web-Showcase/
├── RealObj/
│   └── Components/

```

```
|   └─ Main-Program-ESP8266/
|   └─ Reference/
|   └─ unity/
|       └─ 2 in 1/
|       └─ dashboard/
|       └─ pointer/
|       └─ Prefabs/
|       └─ Scripts/
|   └─ QR-code-CASA0019-Group-Storm.png
|   └─ README.md
```

The above project structure ensures:

Functional partitions: categorised by design tools (e.g. Blender and Fusion), development environments (e.g. Unity and ESP8266), making it easy for developers to quickly locate relevant files.

Content partitioning: separates images, hardware files and reference materials to ensure clear resource management.

Separation of code and design: Hardware, software, 3D models and scripts are categorised to help avoid confusion when collaborating with multiple people.

Team Operation

Tools used for cooperation

Firstly, all development teams use Github for project management based on at least the following two advantages:

1. Source code version tracking: With Git support, the development team can record every change in the project, making it easy to trace back and restore.
2. FREE to use: Individual developers as well as open-source projects can use most of the features free of charge.

Next, the development team chose to use WeChat and Teams as a way to communicate and collaborate remotely

1. WeChat's strengths include high popularity, instant messaging that provides fast and efficient instant messaging, suitable for lightweight discussions or ad-hoc notifications.
2. Teams is suited for online meetings, where sharing screens allows for a more intuitive exchange of ideas (especially when setting project goals together).

Allocation of project tasks

In the earliest stages of the project, Ethan played a very important role in providing an idea for the project and documenting the results of communication with the supervisor. Although Ethan's leaving in the middle of the project was a bit frustrating for the group, I am happy to say that the rest of the group carried on Ethan's ideas and creativity and succeeded in achieving the goals that we set out to achieve:

Dankao Chen:

Design of the exterior of the example in Fusion (shape and structure), printing of most of the components, adjustments to the design of the components (sanding), selection of components, writing of the code for Arduino (physical object), design of the comfort index, writing of the GitHub homepage, displaying the content of the website (codepen)

Muhan Chen:

Blender to Unity, Dashboard design in Unity, Unity to APP, Showcase website content(codepen)

Ying Wu:

Surface design, pointer design, fusion to Blender, Texture design in blender, part of 3D printing work, writing of the GitHub homepage

Although the division of labour does not appear to be the same number of tasks in terms of descriptions, our group has a reasonable division of labour and clear responsibilities, considering the different levels of difficulty of the tasks. Through the above division of labour, team members not only completed their respective tasks well, but also managed to communicate well with each other to ensure the synergistic development of the project.

Limitations

1. Sensor Accuracy and Maintenance

The reliability of the collected data depends heavily on the precision of the hardware sensors. Factors such as sensor quality, placement, and maintenance frequency can significantly influence the accuracy of temperature, humidity, and wind speed readings. If the sensors are not installed correctly or calibrated on a regular basis, the recorded data may deviate from actual conditions, thereby affecting the accuracy of the device's index.

2. Complexity of Feels-Like Temperature Calculation

The feels-like temperature is not solely determined by temperature, humidity, and wind speed. Other important variables—such as solar radiation, an individual's metabolic rate, and personal physical differences—also come into play. Failing to account for these factors may result in less accurate guidance.

3. Variations in Individual Temperature Sensitivity

People differ in how sensitive they are to temperature changes, meaning there is no one-size-fits-all solution

People differ in how sensitive they are to temperature changes, meaning there is no one-size-fits-all solution. While the device can offer general guidelines, personal comfort ranges vary widely; what is comfortable for one person may be too warm or too cool for another. Consequently, the clothing index may not be equally applicable to everyone.

Future Works (Trends)

1. Incorporating Additional Meteorological Parameters

Beyond temperature, humidity, and wind speed, consider integrating more external weather data such as UV index, air pressure, and air quality (e.g., PM2.5, PM10). For example, when the UV index is high, the system could prompt users to wear sun-protective clothing or a hat; when air quality is poor, users might be advised to wear a mask or reduce strenuous outdoor activities. By gathering more comprehensive weather parameters, clothing recommendations become more holistic, providing users with a broader range of health and comfort insights.

2. Add a virtual Avatar

Incorporating a virtual figure or 3D model that can change outfits on-screen offers a more intuitive way of showing which clothing items or combinations are appropriate for the current clothing index.

Reference

Steadman, R.G. (1984). A Universal Scale of Apparent Temperature. *Journal of Climate and Applied Meteorology*, 23(12), pp.1674–1687. doi:[https://doi.org/10.1175/1520-0450\(1984\)023%3C1674:ausoat%3E2.0.co;2](https://doi.org/10.1175/1520-0450(1984)023%3C1674:ausoat%3E2.0.co;2).

Climate of London: https://en.wikipedia.org/wiki/Climate_of_London